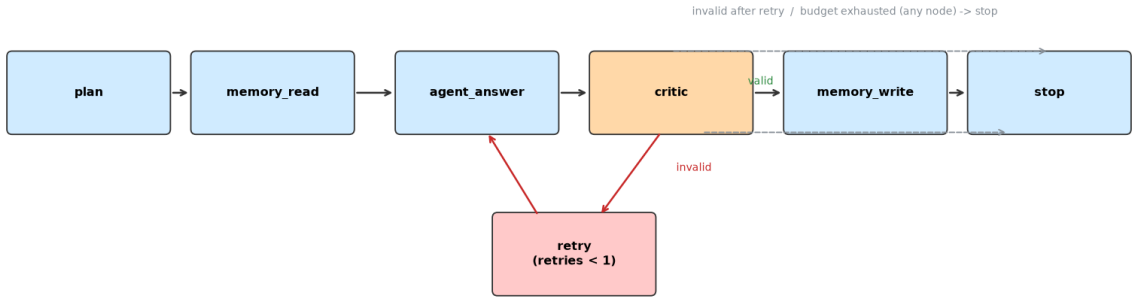
Policy Assistant Agent - Analysis Summary

Week 27 Graded Mini Project · Arpit Jain · Model: gpt-4o-mini (temperature 0) · Full details: notebook, repository and full report

1. What was built

A single tool-calling agent that answers employee questions **only** from an 8-document local policy corpus (data/policies.jsonl, one deliberately superseded version), orchestrated by a LangGraph state machine with a step budget, a deterministic critic and a FAISS memory of verified facts. Modules: models (pydantic schemas, typed errors), tools (policy_search over FAISS, read_policy with a typed PolicyNotFoundError returned as a soft error, safe calculator), prompts (acceptance rules), agent (ReAct AgentExecutor, max 6 iterations, verbose trace), memory, graph, eval, app (CLI).



Graph: plan → memory_read → agent_answer → critic → memory_write → stop; one retry on critic failure; budget router after every node.

2. Acceptance results

Run	Pass rate	Has doc_id*	Has quote*	Avg tool steps	Avg latency (s)	Avg tokens	Memory hits / writes
V1: agent only (no graph/memory/critic)	100% (8/8)	100% (7/7)	100% (7/7)	2.00	3.56	2492	- / -
Graph, cold run (memory cleared)	100% (8/8)	100% (7/7)	100% (7/7)	2.25	3.89	2821	0 / 7
Graph, warm run (memory kept)	100% (8/8)	100% (7/7)	100% (7/7)	1.25	2.88	1572	7 / 0

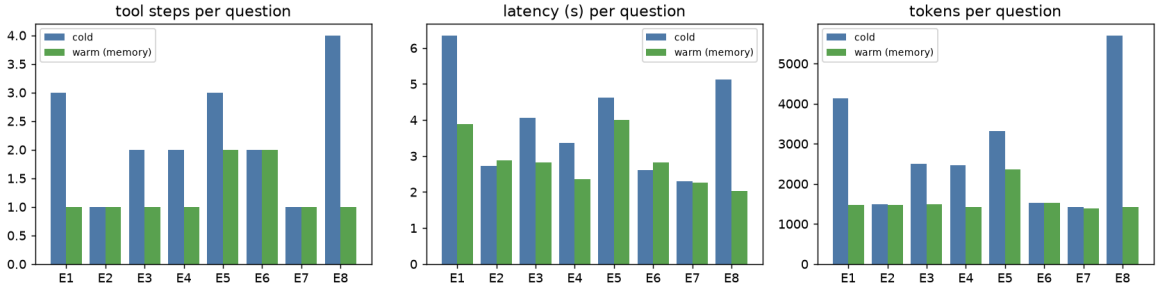
id	kind	question	cited	tool steps (cold→warm)	PASS
E1	factual	What is our remote work limit? Cite and quote.	remote_work_v2	3 → 1	PASS
E2	factual	How many annual leave days and can they be carried over? Cite and quote.	leave_v3	1 → 1	PASS
E3	factual	What's the travel meals cap (INR) and any receipt rule? Cite and quote.	travel_v4	2 → 1	PASS
E4	factual	Is MFA required for VPN? Cite and quote.	security_v1	2 → 1	PASS
E5	recovery	Read policy remote_work_v9.9 and answer the limit.	remote_work_v2	3 → 2	PASS
E6	multi-tool	Compute 12 * 4 and then tell me meals cap with citation.	travel_v4	2 → 2	PASS
E7	factual	What is the minimum password length? Cite and quote.	security_v1	1 → 1	PASS
E8	boundary	What is the CEO's annual salary?	-	4 → 1	PASS

Acceptance target (≥ 80% of queries pass): **met**. Pass = critic-approved answer that cites the expected doc_id with a verbatim quote and contains the key facts (boundary question: refuses and cites nothing). *Over the 7 answerable questions; the boundary question correctly has neither. Tests: 22 passed in 11.27s.

3. Analysis

- **Grounding.** Every factual answer carries a [doc_id] token and a short verbatim quote; the critic re-checks both against the corpus, rejects numbers that do not appear in the cited policy, and rejects the superseded remote_work_v1. The out-of-corpus question (CEO salary) is refused without citations.
- **Tool use and recovery.** The bad-id case (E5) calls read_policy('remote_work_v9.9'), receives a soft PolicyNotFoundError with suggestions, switches to policy_search and answers from remote_work_v2 in 3 tool steps. The multi-tool case (E6) uses the calculator for 12 × 4 = 48 and then cites the meals cap. All runs stay within max_iterations (6) and the graph budget (8 nodes); 1 critic retry was needed in the cold and warm evaluations.
- **Memory.** Only critic-approved answers with existing doc_ids and verbatim quotes are written (7 writes on the cold run; the refusal is not stored). On the warm run 7/8 questions hit memory and the agent confirms the remembered doc_id with one read_policy instead of searching: average tool steps fall by 44% and tokens by 44%; latency fell from 3.89s to 2.88s.

- **Orchestration.** Explicit nodes and routers make every run traceable (node trace in each log); controlled tests show the budget stop (max_steps = 3 stops before the critic) and exactly one retry before failed_critic, with no memory write. Optional SQLite checkpointing persists each node's state.



Per-question tool steps, latency and tokens: cold vs warm run.

4. Limitations and next steps

- Tiny synthetic corpus (8 policies) and similarity thresholds tuned on it; the deterministic critic checks quotes and numbers but not paraphrased claims (the optional LLM critic covers this at extra cost).
- Next: per-node timeouts, a planner/solver/critic multi-agent variant, SQLite-table memory with expiry when a policy version changes, and a larger regression set of real employee questions.

Reproduce: install requirements.txt, put OPENAI_API_KEY in agentic-policy-assistant/.env, run pytest and then the notebook (see the README). app.py --offline runs everything without a key.